A REALISTIC IMAGE GENERATION OF FACE FROM TEXT DESCRIPTION USING GENERATIVE ADVERSARIAL NETWORKS

A Project Report

Submitted By

THOTA SAVEEN KUMAR
2303031241489

S.MD. TAHER HUSSAIN
2303031241306

S. BHARATH REDDY
2303031241282

A. VARUN
2303031241617

in Partial Fulfilment for the Award of the Degree of

BACHELOR OF TECHNOLOGY

COMPUTER SCIENCE & ENGINEERING

Under the Guidance of

Binisa Makwana

Assistant Professor



VADODARA

October - 2026



PARUL UNIVERSITY

CERTIFICATE

This is to certify that Project - II (303105423) of 7th Semester entitled **"A Realistic Image Generation of Face from Text Description Using Generative Adversarial Networks"** of Group No. **NON-ASMP_AI-113** has been successfully completed by:

Thota Saveen Kumar	-	**2303031241489**
S.MD. Taher Hussain	-	**2303031241306**
S. Bharath Reddy	-	**2303031241282**
A. Varun	-	**2303031241617**

under my guidance in partial fulfilment of the Bachelor of Technology (B.Tech) in Computer Science & Engineering of Parul University in Academic Year 2025-2026.

Date of Submission: _____

Asst. Prof. Binisa Makwana
Project Guide

Dr. Sanjay Agal
Head of Department, AIDS, PIET,
Parul University NAAC A++

Dr. Mohammad Arif
Project Coordinator

Acknowledgements

We take this opportunity to express our sincere gratitude to all those who supported us in the successful completion of this project. First and foremost, we extend our heartfelt thanks to our project guide, Asst. Prof. Binisha Makwana, for her constant encouragement, valuable suggestions, and expert guidance. Her patience and motivation have been a source of inspiration throughout this work.

We are also grateful to our teachers and faculty members, who provided the direction and academic knowledge that contributed substantially to the development of this project. We thank our Head of the Department, Dr. Sanjay Agal, and our Project Coordinator, Dr. Mohammad Arif, for their support. We further acknowledge Parul University for providing the facilities, resources, and an excellent learning environment in which this work was carried out. We particularly appreciate the cooperation, constructive feedback, and encouragement offered by our classmates and friends at various stages of the project. We also extend our gratitude to the authors, researchers, and members of the open-source community whose work and resources assisted us in completing this project.

Finally, we express our deepest appreciation to our parents and family members for their continuous support, understanding, and encouragement. Their constant confidence in our abilities has been a constant source of motivation. We also acknowledge the contribution of all individuals who, directly or indirectly, played a role in the completion of this project. This project has been a valuable learning experience, and we respectfully dedicate its outcome to everyone who made it possible.

Thota Saveen Kumar
S.MD. Taher Hussain
S. Bharath Reddy
A. Varun
B.Tech. – CSE (AI)
Parul University, Vadodara

Abstract

The generation of realistic facial images from textual descriptions has become increasingly significant in artificial intelligence and computer vision because of its applications in digital content creation, forensic sketching, entertainment, and virtual avatars. This project, titled *A Realistic Image Generation of Face from Text Description Using Generative Adversarial Networks* (GANs), presents a methodology for synthesising photorealistic human faces from natural-language input. The proposed system follows a two-stage architecture. In the first stage, the textual description is processed with natural-language processing techniques to extract semantic features and produce an embedding. In the second stage, a GAN-based image generator converts this embedding into a facial image.

In contrast with traditional generative models such as Variational Autoencoders (VAEs), which frequently produce low-resolution or blurred results, adversarial training with deep convolutional networks improves image clarity, feature accuracy, and diversity. The generator and the discriminator refine the output iteratively so that it corresponds more closely to the described attributes. The report covers the literature survey, the system analysis, the design (including use case, activity, sequence, and class diagrams together with flowcharts), the methodology, the implementation using Python-based tools such as TensorFlow, PyTorch, OpenCV, Hugging Face libraries, and Flask, and the testing approach. The prototype provides an interactive web interface in which a user enters a description and receives a generated face. By integrating text-to-image mapping with GAN architectures, the work offers a flexible and scalable approach from descriptive language to realistic human face images.

Keywords: Generative Adversarial Networks, Text-to-Image Synthesis, Face Generation, CelebA, Natural Language Processing, Deep Learning.

Table of Contents

Certificate ..... **i**

Acknowledgements ..... **ii**

Abstract ..... **iii**

List of Figures ..... **viii**

List of Tables ..... **ix**

Chapter 1: Introduction **1**

1.1 Background 1

1.2 Problem Statement 1

1.3 Objectives 1

1.4 Scope of the Project 2

1.5 Organisation of the Report 2

Chapter 2: Literature Survey **4**

2.1 Introduction 4

2.2 Review of Existing Work 4

2.3 Comparative Analysis 6

2.4 Discussion 7

Chapter 3: System Analysis **9**

3.1 Introduction 9

3.2 Functional Requirements 9

3.3 Non-functional Requirements 10

3.4 System Components 10

3.5 System Constraints 11

3.6 Hardware and Software Requirements 11

3.7 Use Case Diagram 11

3.8 Summary and Next Steps 12

Chapter 4: System Design **13**

4.1 Introduction 13

4.2 High-Level System Architecture 13
4.3 Data Flow 14
4.4 Text Encoder and GAN Design 14
4.5 UML Diagrams 15
4.6 Security, Privacy and Ethical Considerations 17
4.7 Scalability Considerations 18
4.8 Implementation Challenges and Solutions 18
4.9 Summary 18
Chapter 5: Methodology 19
5.1 System Overview 19
5.2 Dataset 20
5.3 Text Description Preparation and Embedding 20
5.4 Conditional GAN 20
5.5 Training Procedure 21
5.6 Inference Procedure 21
5.7 Evaluation Strategy 22
5.8 Deployment Plan 22
Chapter 6: Implementation 23
6.1 Introduction 23
6.2 Development Environment 23
6.3 Dataset Preparation 24
6.4 Model Training 24
6.5 Web Application 24
6.6 Generation Back End 25
6.7 Sample Output 26
6.8 Implementation Challenges 27
6.9 Summary 27
Chapter 7: Testing 28
7.1 Introduction 28
7.2 Test Strategy 28

7.3 Functional Test Cases 28
7.4 Image Quality Evaluation 29
7.5 Observations from the Prototype 29
7.6 Limitations of the Study 29
7.7 Summary 30
Chapter 8: Conclusion 31
8.1 Conclusion 31
8.2 Achievement of Objectives 32
8.3 Key Contributions 32
8.4 Limitations Observed 33
8.5 Ethical and Social Perspective 33
8.6 Future Enhancements 33
8.7 Final Remarks 34
Chapter 9: Future Work 35
9.1 Stronger Text Understanding 35
9.2 Rare and Fine-Grained Attributes 35
9.3 Higher Resolution and Realism 35
9.4 Interactive Editing and Refinement 36
9.5 Fairness, Bias, and Responsible Use 36
9.6 Multilingual Prompts 36
9.7 Performance and Deployment 36
9.8 Validation with Domain Experts 37
9.9 Beyond Static Images 37
9.10 Summary of Future Directions 37
References 38
Appendix A: Installation and Deployment Guide 42
A.1 Prerequisites 42
A.2 Python Packages and Environment Variable 42
A.3 Start-up Steps 43
A.4 Troubleshooting 43

Appendix B: Sample Prompt Templates 44

B.1 Example Prompts 44

B.2 Attribute-to-Caption Mapping for Training 44

Appendix C: Glossary 46

List of Figures

Figure 1.1: Overview of the text-to-face generation workflow 2
Figure 2.1: Maximum reported output resolution of the reviewed models 7
Figure 2.2: Distribution of the 20 reviewed works by category 7
Figure 3.1: Use case diagram of the text-to-face generation system 12
Figure 4.1: Layered architecture of the text-to-face generation system 13
Figure 4.2: Data flow from text description to generated face 14
Figure 4.3: Activity diagram of the system 15
Figure 4.4: Sequence diagram of the text-to-face generation process 16
Figure 4.5: Class diagram of the system 17
Figure 5.1: Flowchart of the overall methodology 19
Figure 5.2: Adversarial training loop of the conditional GAN 21
Figure 5.3: Flowchart of the inference (face generation) process 22
Figure 6.1: Project structure and application code in Visual Studio Code 23
Figure 6.2: Home page of the GAN Face Generator web application 25
Figure 6.3: Model loading and Flask server start-up in the terminal 26
Figure 6.4: Generated face displayed in the web application 26

List of Tables

Table 2.1: Comparison of representative text-to-image and face-generation models 6
Table 3.1: Functional requirements 9
Table 3.2: Hardware and software requirements 11
Table 4.1: Implementation challenges and solutions 18
Table 7.1: Functional test cases 28
Table 7.2: Evaluation metrics 29
Table 8.1: Project objectives and their status 32
Table 9.1: Planned future work and priorities 37
Table A.1: Required packages and configuration value 42
Table A.2: Common problems and remedies 43
Table B.1: Example face descriptions 44
Table B.2: CelebA attributes and caption phrases 45
Table C.1: Glossary of key terms 46

Chapter 1

Introduction

1.1 Background

Artificial Intelligence (AI) has achieved rapid progress in the domains of computer vision and natural language processing. One of the most promising areas of research is text-to-image synthesis, in which the objective is to generate meaningful images directly from natural-language descriptions. The ability to generate a human face from descriptive text has broad applications in forensics, gaming, the film industry, virtual avatars, and assistive AI tools.

Generative Adversarial Networks (GANs), introduced by Goodfellow et al. in 2014 [1], have become the foundation for realistic image generation. A GAN consists of two networks, a Generator and a Discriminator, which are trained in opposition to each other. The generator produces increasingly realistic images, while the discriminator learns to distinguish them from real images. By conditioning a GAN on text embeddings [3], it becomes possible to synthesise face images that align with descriptive features such as age, gender, hair type, and facial accessories.

1.2 Problem Statement

Traditional face-generation methods require either three-dimensional modelling or manual sketching by a trained artist. Both approaches are time-consuming and frequently lack accuracy. Current research seeks to automate this process by employing GANs conditioned on textual descriptions. The principal challenge lies in ensuring that the generated faces are both photorealistic and consistent with the given description. A face that appears real but disregards the requested attributes is of limited practical value.

1.3 Objectives

The principal objectives of this project are as follows:

1. To study and analyse existing GAN-based text-to-image methods.

2. To design and implement a GAN framework capable of generating human face images from natural-language descriptions.
3. To evaluate the generated images with respect to realism and semantic accuracy.
4. To provide an interactive interface for end users.

1.4 Scope of the Project

This project focuses exclusively on face generation from text descriptions. It employs deep-learning models to embed text inputs and integrates them with a conditional GAN to produce images. The principal application areas are as follows:

- **Forensics:** generating faces from witness descriptions.
- **Entertainment:** creating avatars for games and films.
- **Research:** studying the integration of multi-modal deep-learning techniques.

The scope is limited to static two-dimensional face images with basic features such as age, gender, hairstyle, and accessories. Rare high-level attributes, such as scars and tattoos, are considered extensions for future work. Figure 1.1 provides an overview of the intended workflow. A text description is processed to extract facial attribute values, these values manipulate a latent code, and a synthesis network converts the final latent vector into a face image.

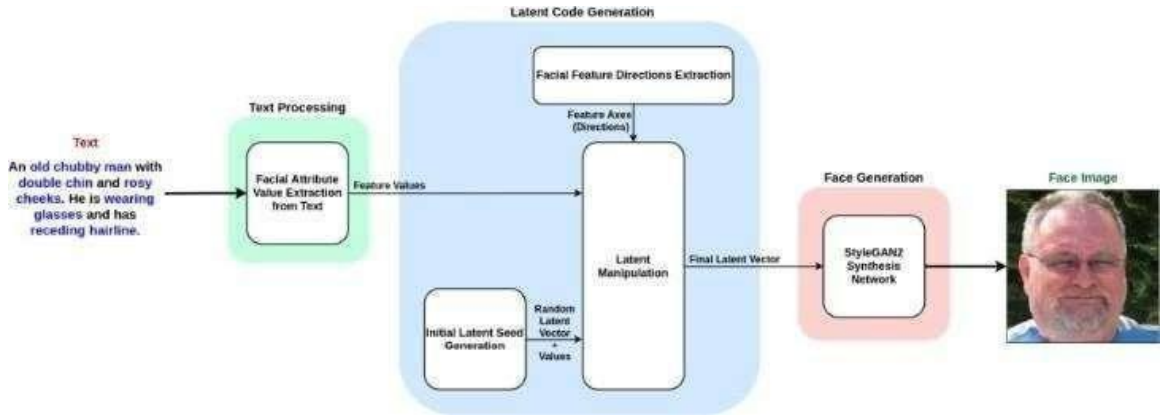


Figure 1.1: Overview of the text-to-face generation workflow

1.5 Organisation of the Report

The report is organised as follows:

- Chapter 1 introduces the project, its objectives, and its scope.
- Chapter 2 presents a literature survey covering 20 related works.
- Chapter 3 details the system analysis and the requirements specification.
- Chapter 4 describes the system design, including diagrams and flowcharts.
- Chapter 5 explains the methodology adopted.
- Chapter 6 covers the detailed implementation steps.
- Chapter 7 discusses the testing strategy and the observations made.
- Chapter 8 concludes the findings of the project.
- Chapter 9 identifies directions for future work.
- The References section lists the sources in IEEE format.
- Appendices A to C provide the installation and deployment guide, sample prompt templates, and a glossary of key terms.

Chapter 2
Literature Survey

2.1 Introduction

The generation of realistic images from textual descriptions has been an active area of research in recent years. Generative Adversarial Networks and their extensions have served as the principal foundation for such tasks. This chapter reviews 20 significant research works relevant to text-to-image synthesis and face generation, and it highlights their methodologies, advantages, and limitations.

2.2 Review of Existing Work

Foundations: GANs and text conditioning

Generative Adversarial Networks were first proposed by Goodfellow et al. (2014) [1], in which a generator and a discriminator compete to produce realistic synthetic images. This method was groundbreaking, but it did not support conditional inputs such as text. Radford et al. [2] made GAN training practical for images through deep convolutional architectures (DCGAN). Reed et al. (2016) [3] subsequently demonstrated text-conditioned GANs by employing a DCGAN with text embeddings, generating small images of birds and flowers. In the same year, Reed et al. [4] explored joint text-image embedding models for fine-grained visual descriptions. Neither work, however, achieved realistic results for human faces.

Multi-stage and attention-based text-to-image GANs

Zhang et al. (2017) introduced StackGAN [5], which refined image quality in two stages to produce outputs of higher resolution. Its successor, StackGAN++ [6], improved photorealism through a multi-branch structure, but it required substantial computation and larger datasets. Xu et al. (2018) advanced the field with AttnGAN [7], which incorporated attention mechanisms to align words with image regions. AttnGAN produced sharper and semantically more relevant 256 × 256 images, although it exhibited training instability and high GPU requirements. DM-GAN, proposed by Zhu et al. (2019) [8], added a dynamic memory module to refine poorly generated regions. MirrorGAN, proposed by Qiao et al.

(2019) [9], combined text-to-image and image-to-text cycles to enhance semantic alignment. StoryGAN [10] generated sequences of images for story-based descriptions. The outputs of these models, however, still lacked realistic facial synthesis.

Image manipulation and translation

Zhu et al. (2016) proposed iGAN [12], which enabled interactive editing of images on the learned image manifold. Nam et al. (2018) proposed TAGAN [11] to edit existing images using text. Zhu et al. (2017) developed CycleGAN [17] for unpaired image-to-image translation. These methods modify existing pictures rather than synthesise a new face from a description.

High-fidelity face generation

Karras et al. introduced Progressive GANs [13], which grow the resolution gradually during training in order to produce ultra-realistic 1024 × 1024 human faces. Building on this foundation, StyleGAN [14] (2019) added a style-based generator with style mixing for fine-grained attribute control, and StyleGAN2 [15] (2020) removed characteristic artefacts to yield more stable results. BigGAN [16] demonstrated that scaling GANs on large datasets produces diverse, high-resolution images. Although these models excel at generating realistic human faces, they are not conditioned on text.

Transformer and diffusion-based approaches

More recent approaches have shifted towards transformers and diffusion models. Ramesh et al. (2021) introduced DALL·E [18], a transformer-based model capable of creative open-domain image generation, although it is highly resource-intensive. Diffusion models such as GLIDE [19], which employs classifier-free guidance, Imagen [20], which achieves near-human-level realism, and Stable Diffusion [21], which applies latent diffusion and is efficient and open source, have established a new benchmark. Nevertheless, even the most advanced models occasionally fail to capture fine-grained facial details.

Work on text-to-face generation

Specific work on faces includes Text2FaceGAN, proposed by Nasir et al. [22], which generates faces from fine-grained textual descriptions, and the Semantic Text-to-Face GAN (ST2FG) proposed by Oza et al. [23], which incorporates semantic information from the text to improve identity and attribute consistency. Other studies apply conditional and deep convolutional GANs to the same problem [24], [25]. Recent surveys [26], [27] and a

review of StyleGAN-based methods [28] summarise the progress and the open challenges in this area.

2.3 Comparative Analysis

The reviewed literature indicates that early GAN-based models provided the initial proof of concept but were limited in realism and resolution. Subsequent works, such as StackGAN and AttnGAN, improved image quality and semantic alignment but encountered difficulties with training stability and hardware requirements. Progressive GAN and StyleGAN achieved highly realistic face synthesis, although they did not incorporate text conditioning. Diffusion-based approaches, particularly Imagen and Stable Diffusion, currently represent the state of the art by combining efficiency, scalability, and high-quality results. Table 2.1 compares the principal models, and Figures 2.1 and 2.2 summarise the maximum reported output resolution and the distribution of the reviewed works.

Table 2.1: Comparison of representative text-to-image and face-generation models

Model (Year)	**Key idea**	**Text input**	**Face focus**	**Principal limitation**
GAN (2014) [1]	Generator versus discriminator	No	No	No conditioning
Reed et al. (2016) [3]	DCGAN with text embedding	Yes	No	64 × 64 output; birds and flowers
StackGAN (2017) [5]	Two-stage refinement	Yes	No	Heavy computation
AttnGAN (2018) [7]	Word-region attention	Yes	No	Unstable training; high GPU requirement
ProGAN (2017) [13]	Progressive growing	No	Yes	No text control
StyleGAN / StyleGAN2 [14], [15]	Style-based generator	No	Yes	No text control
Text2FaceGAN (2019) [22]	GAN from face descriptions	Yes	Yes	Limited resolution
Stable Diffusion (2022) [21]	Latent diffusion	Yes	General	Fine facial detail errors; large model

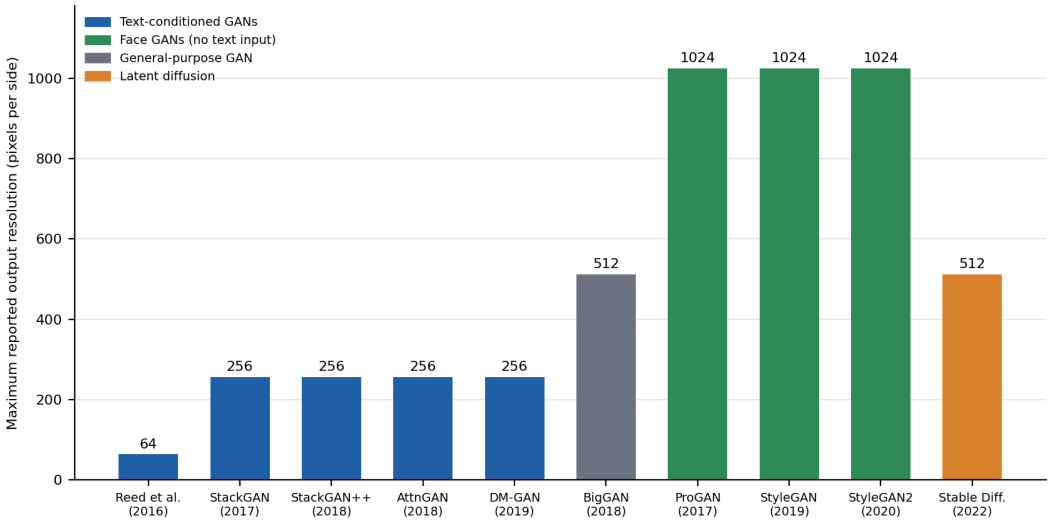


Figure 2.1: Maximum reported output resolution of the reviewed models

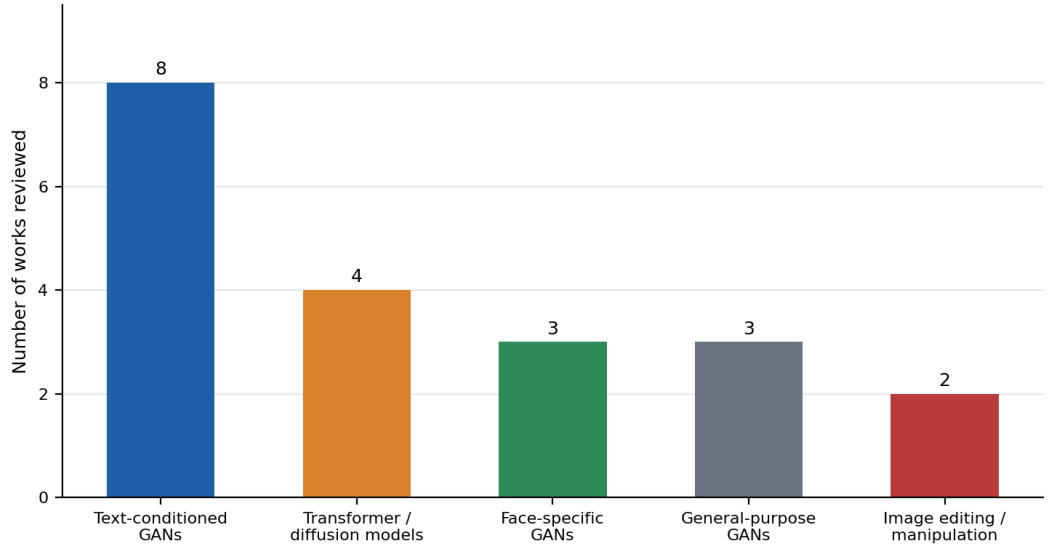


Figure 2.2: Distribution of the 20 reviewed works by category

2.4 Discussion

The literature survey highlights the rapid evolution of text-to-image synthesis methods. Early efforts established feasibility but lacked photorealism, whereas multi-stage and attention-based GANs considerably improved quality and semantic accuracy. StyleGAN and Progressive GAN transformed face generation with unmatched realism, although they were not designed for text conditioning. Diffusion models have since set a new benchmark

by producing human-level images directly from text prompts. Figure 2.1 illustrates this trend clearly: text-conditioned GANs remained at or below 256 pixels, while face-specific GANs reached 1024 pixels without accepting text.

The research gap is therefore the combination of two strengths: face-specific realism and fine-grained text control. This project builds upon these advancements by focusing on the generation of faces specifically from textual descriptions, using GANs conditioned on embeddings. Unlike prior models, which were either domain-specific or lacked text-based control, the proposed approach aims to bridge this gap, with potential applications in forensics, entertainment, and avatar creation.

Chapter 3

System Analysis

3.1 Introduction

The System Requirements Specification (SRS) outlines the functional and non-functional requirements of the proposed text-to-face generation system. The system accepts a natural-language description of a human face and returns a synthesised face image through an interactive web interface. This chapter describes the principal requirements, the system components, the constraints, and the hardware and software environment required to build and operate the system. It also presents the use case diagram that captures the interaction between the user and the system.

3.2 Functional Requirements

The functional requirements define the essential tasks that the system must support. They are summarised in Table 3.1.

Table 3.1: Functional requirements

ID	Requirement	Description
FR1	Dataset upload	The system shall allow the CelebA face dataset and its attribute annotations to be loaded for training.
FR2	Text input	The user shall be able to enter a free-form description of a face (age, gender, hair, eyes, accessories).
FR3	Text processing	The system shall tokenise the description, extract facial attributes, and convert them into an embedding vector.
FR4	Model training	The system shall train the conditional GAN so that the generator learns to match the described attributes.
FR5	Image generation	The system shall generate a face image conditioned on the processed text and a random noise vector.
FR6	Result display	The system shall display the generated face in the browser and allow it to be downloaded or printed.

ID	Requirement	Description
FR7	History	The system shall maintain a generation history so that earlier results can be revisited.
FR8	Input validation	The system shall reject empty or unusable input and display a clear message.

3.3 Non-functional Requirements

The non-functional requirements define the performance, usability, and reliability expectations of the system.

- **Performance:** an image should be generated within a few seconds on a GPU-enabled machine. CPU-only operation is supported but considerably slower.
- **Quality:** the generated faces should appear photorealistic and be consistent with the attributes stated in the prompt.
- **Usability:** the interface should be simple, comprising a single text box, a generate button, and a clear result area.
- **Reliability:** the application should handle invalid input and model-loading errors without terminating.
- **Portability:** the system should operate on Windows and Linux with a standard Python environment.
- **Maintainability:** the model, text-processing, and interface code should be separated into modules so that each can be improved independently.
- **Ethics and privacy:** the system should generate synthetic faces only, should not store personal data, and should be used responsibly (see Section 4.6).

3.4 System Components

The system comprises four principal components. The Text Processing Module tokenises the description of the user, extracts attributes such as age, gender, hair colour, and accessories, and forms a text embedding. The Text Encoder maps the processed text into a latent conditioning vector that the generator can utilise. The GAN Module contains the generator, which synthesises images from the noise vector and the conditioning vector, and

the discriminator, which is employed during training to determine whether an image is real and whether it matches the text. The Web Interface collects the prompt, triggers the generation process, and displays the result together with the generation history.

3.5 System Constraints

Despite its capabilities, the system is subject to several inherent constraints. The quality of the generated output depends on the size and diversity of the training data. CelebA consists predominantly of celebrity faces, which introduces demographic bias. GAN training is sensitive to hyper-parameters and may suffer from mode collapse or instability. Very long or ambiguous descriptions may not be fully represented in the output, and rare attributes, such as scars or tattoos, are not covered. The speed of generation depends strongly on the available GPU.

3.6 Hardware and Software Requirements

Table 3.2: Hardware and software requirements

Item	Recommended specification
Processor	Intel Core i5 / AMD Ryzen 5 or higher
RAM	8 GB minimum; 16 GB recommended
GPU	NVIDIA GPU with CUDA support and 6 GB or more of VRAM (for training and efficient inference)
Storage	20 GB or more of free space for datasets, model weights, and outputs
Operating system	Windows 10/11 or Ubuntu 20.04 and above
Languages and libraries	Python 3, TensorFlow, PyTorch, OpenCV, NumPy, Hugging Face Transformers and Diffusers
Web framework	Flask, with HTML, CSS, and JavaScript for the front end
Tools	Visual Studio Code and a modern web browser

3.7 Use Case Diagram

Figure 3.1 presents the use case diagram. The user uploads the CelebA dataset and enters a text description to generate an image, while the GAN model produces the fully trained

model and generates the image.

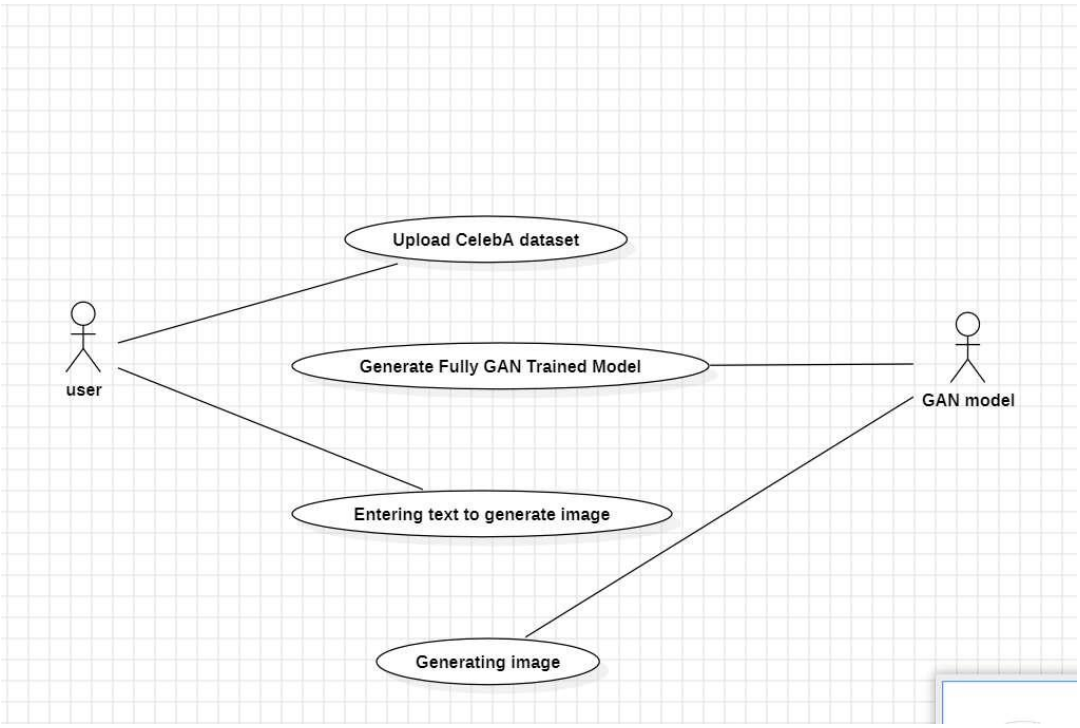


Figure 3.1: Use case diagram of the text-to-face generation system

3.8 Summary and Next Steps

The SRS provides an outline of the functional and non-functional requirements and confirms that the text-to-face system is technically sound and usable. The next step is to define the detailed system architecture and data flow, to select the model structure, and to commence implementation in a phased manner. The phases are dataset preparation, followed by the text encoder, then GAN training, and finally the web interface.

Chapter 4

System Design

4.1 Introduction

The design of the system focuses on generating realistic faces that conform to the description provided by the user, while remaining simple to operate. This chapter presents the architecture, the flow of data from text to image, the design of the text encoder and the GAN, and the UML diagrams that describe the behaviour of the system.

4.2 High-Level System Architecture

The architecture is organised into three layers, as shown in Figure 4.1. The **Presentation Layer** is the Flask web interface, in which the user enters a description and views the result. The **Language Processing Layer** tokenises the text, extracts attributes, and produces an embedding. The **Image Synthesis Layer** combines the embedding with a noise vector in the generator to produce the face. The discriminator participates only during training, and OpenCV is employed for post-processing operations such as resizing and colour handling.

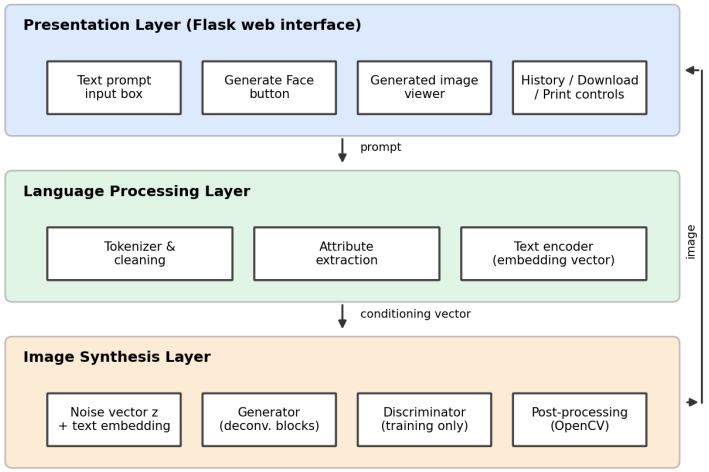


Figure 4.1: Layered architecture of the text-to-face generation system

The three layers operate in a single direction for each request. The presentation layer passes the prompt to the language processing layer, which returns a conditioning vector. The image synthesis layer then combines that vector with a newly sampled noise vector, so that each request can yield a different face that satisfies the same description. Separating the layers in this manner allows each one to be modified or replaced independently. For example, the text encoder may be upgraded to a larger model while the web interface remains unchanged.

4.3 Data Flow

The data flow is depicted in Figure 4.2. The description provided by the user is tokenised and parsed for attributes, and the result is converted into a conditioning vector. The generator receives this vector together with random noise and produces a face image. During training, real CelebA images and generated images are passed to the discriminator, whose real or fake feedback is used to improve the generator.

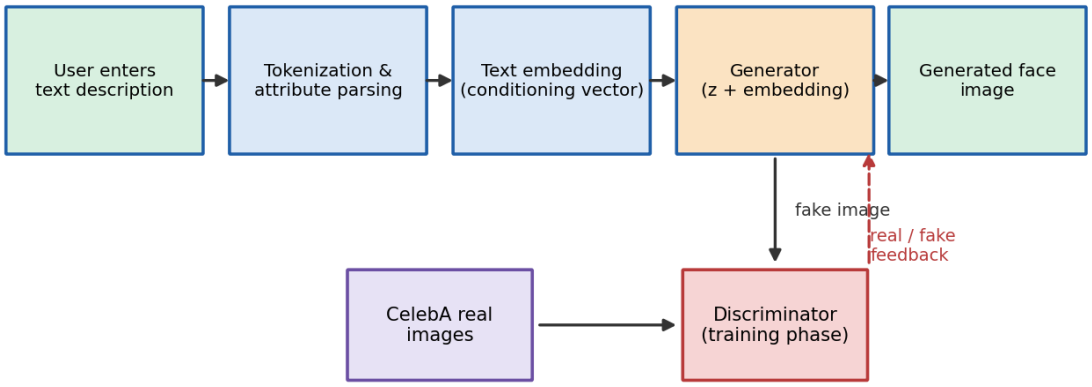


Figure 4.2: Data flow from text description to generated face

4.4 Text Encoder and GAN Design

The **text encoder** converts a sentence into a vector of fixed length. Pre-trained language models, such as transformer-based encoders [33], can be employed so that the system understands varied phrasing, for example "curly hair" and "wavy hair", rather than only fixed keywords. The resulting vector is concatenated with the noise vector before the first

layer of the generator.

The **generator** is a deep convolutional network that up-samples the combined vector through transposed-convolution blocks with batch normalisation and ReLU activations. It concludes with a tanh layer that outputs an RGB face image. The **discriminator** is a convolutional network that down-samples the image and also receives the text embedding, so that it evaluates both realism and agreement with the description. Training follows the standard adversarial objective of Goodfellow et al. [1], extended to a conditional form [3].

4.5 UML Diagrams

Three UML diagrams describe the dynamic and static structure of the system. The activity diagram in Figure 4.3 presents the principal flow of activities: the CelebA dataset is uploaded, the fully trained model is generated, and an image is generated from text.

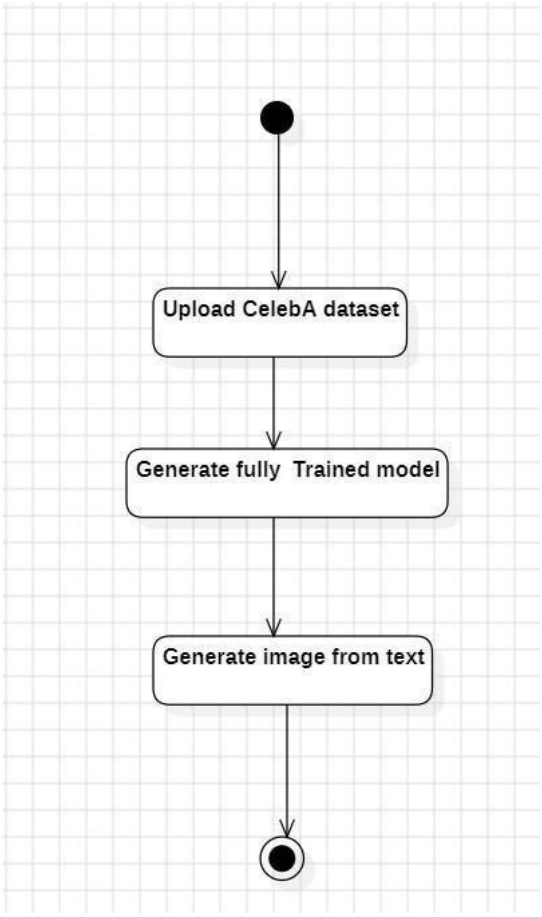


Figure 4.3: Activity diagram of the system

The sequence diagram in Figure 4.4 illustrates the interaction over time between the user and the system components. The user first uploads the dataset and receives a confirmation message. The user then requests the training of the GAN model and, finally, requests an image from a text prompt.

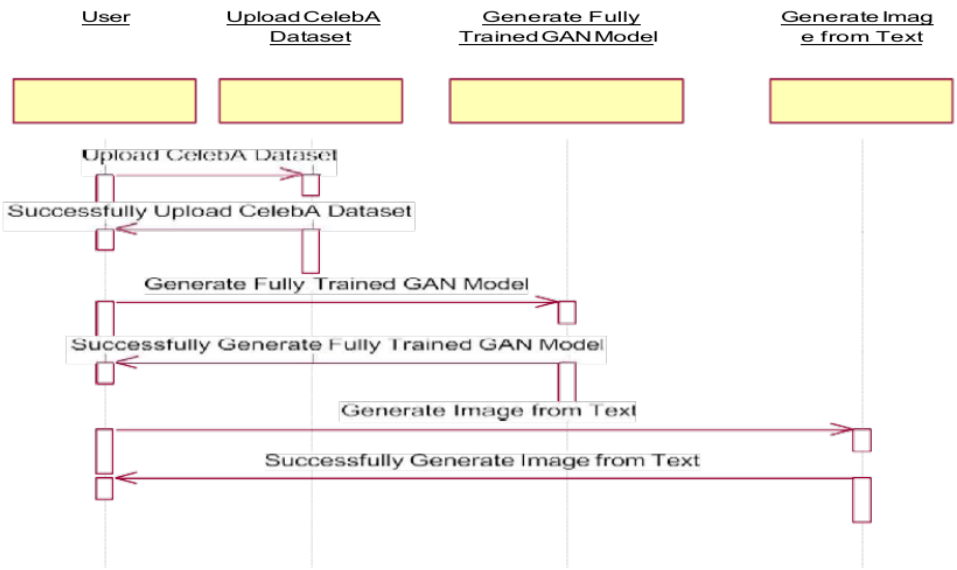


Figure 4.4: Sequence diagram of the text-to-face generation process

The class diagram in Figure 4.5 presents the principal classes. The User supplies text to the TextProcessor, which tokenises and embeds the text. The TextEncoder converts the embedding into a latent vector. The Generator produces the image, which is displayed and saved by ImageOutput. The CelebADataset class loads and pre-processes the real images for the Discriminator, and the GAN class trains the generator and the discriminator jointly.

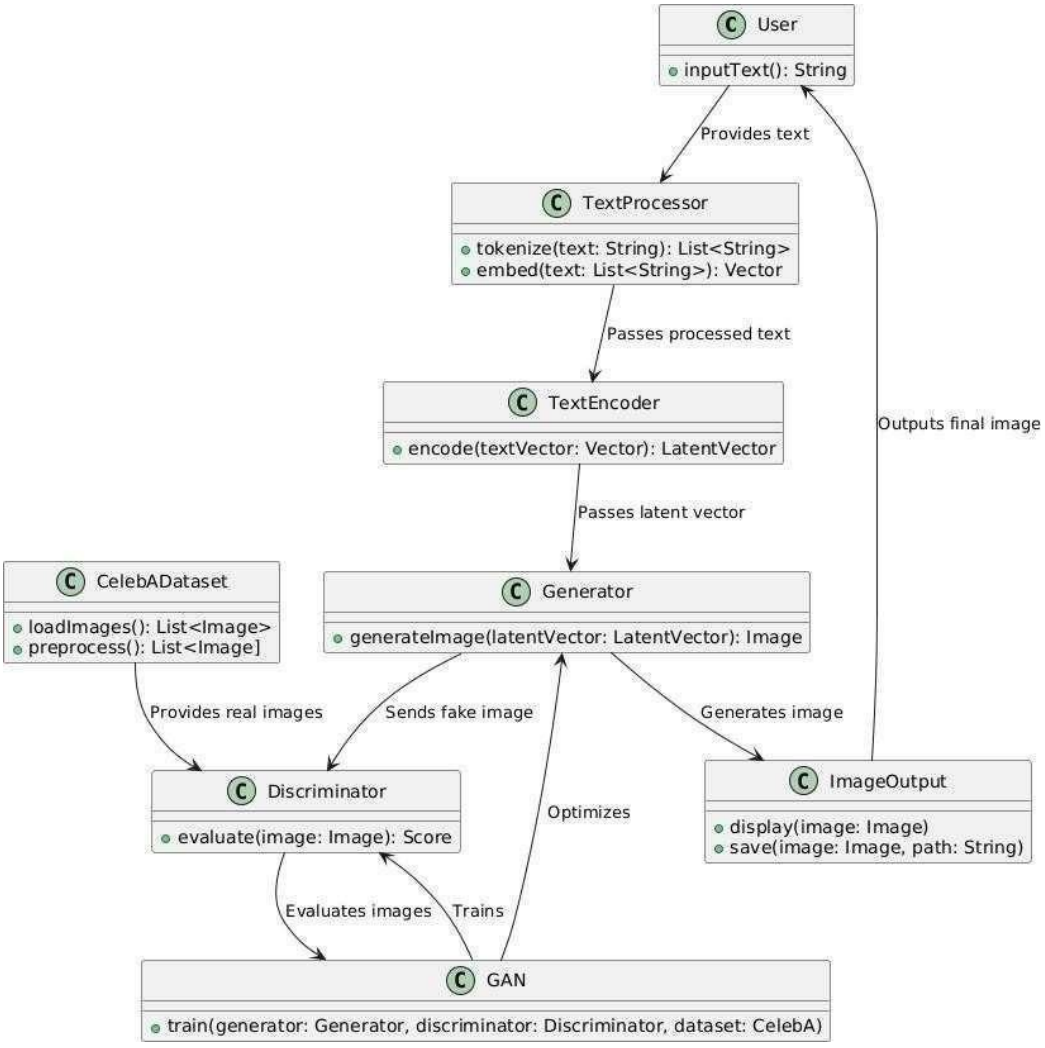


Figure 4.5: Class diagram of the system

4.6 Security, Privacy and Ethical Considerations

Face-generation technology may be misused to create misleading or deceptive content. The system therefore generates only synthetic faces, does not accept or store photographs of real persons, and retains no personal data beyond the local generation history. For forensic applications, a generated face must be treated as an investigative aid and not as evidence of identity. The training data also requires careful attention. CelebA is not demographically balanced, and the outputs may therefore be of higher quality for some groups than for others. Future versions should incorporate an invisible watermark or a visible "AI-generated" label on outputs, and should balance or augment the training set in order to

reduce bias.

4.7 Scalability Considerations

The modular design permits the text encoder, the generator, and the interface to be upgraded independently. The model can be exported and served through a lightweight Flask service for a single user. Alternatively, it can be placed behind a task queue with multiple GPU workers if a large number of users is anticipated. Caching of the loaded model weights avoids repeated start-up delays, and generated images can be written to a storage directory with a simple index that supports the history feature.

4.8 Implementation Challenges and Solutions

Table 4.1 summarises the principal challenges anticipated during implementation, together with the proposed solutions.

Table 4.1: Implementation challenges and solutions

Challenge	Proposed solution
Unstable GAN training and mode collapse	Use batch normalisation, label smoothing, balanced learning rates, and monitor loss curves
Text not reflected in the generated image	Use a matching-aware discriminator and richer attribute captions
Limited image resolution and detail	Adopt progressive resolution growth or a style-based generator in future versions
High GPU demand	Apply mixed-precision computation, smaller batch sizes, and CPU fallback for inference
Dataset bias	Apply balanced sampling and augmentation; report results across attribute groups

4.9 Summary

The design combines a text-processing front end, a conditional GAN, and a web interface within a modular architecture. The diagrams and flowcharts presented in this chapter define the structure and behaviour of the system, and they provide the basis for the methodology described in the following chapter.

Chapter 5

Methodology

5.1 System Overview

The methodology follows the pipeline depicted in Figure 5.1. Face images and their attribute annotations are collected and pre-processed, the attributes are converted into natural-language captions, and the captions are encoded into embeddings. A conditional GAN is then trained on image-caption pairs. When the quality of the output is acceptable, the model is deployed in the web application and evaluated with respect to realism, text-image consistency, and user feedback.

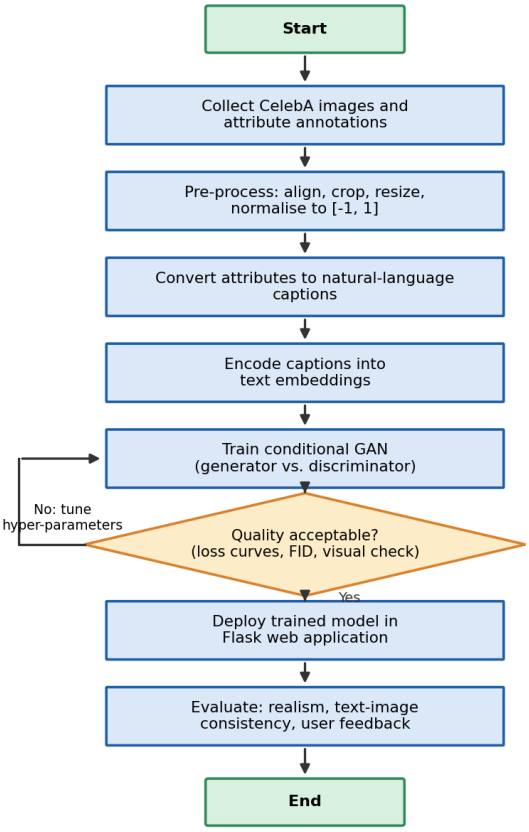


Figure 5.1: Flowchart of the overall methodology

The pipeline is executed in a fixed sequence, and the output of each stage forms the input of the next. The first stage verifies the completeness of the CelebA image and attribute files and discards any records that are corrupted or incomplete. The second stage aligns, crops, and resizes each face and normalises the pixel values to the range [-1, 1]. The third stage converts the binary attribute labels into captions, so that every image is associated with a textual description. The fourth stage encodes the captions into embeddings, which are stored so that they need not be recomputed during training.

Training then proceeds in accordance with the adversarial procedure described in Section 5.5. The loss values, the sample grids, and the FID score are monitored at fixed intervals to determine whether the hyper-parameters require adjustment. Only when the model satisfies the quality criteria is it exported for deployment in the web application.

5.2 Dataset

The CelebFaces Attributes dataset (CelebA) [29] is employed in this project. It contains more than 200,000 celebrity face images, each annotated with 40 binary attributes, including Young, Male, Eyeglasses, Smiling, Blond_Hair, Bangs, and Wearing_Hat. These attributes cover the principal features within the scope of the project, namely age, gender, hairstyle, and accessories. The images are aligned and cropped, and they are then resized and normalised to the range [-1, 1] before training.

5.3 Text Description Preparation and Embedding

CelebA provides attributes rather than sentences. Each image is therefore paired with a caption constructed from its positive attributes using templates, for example, "A young woman with blond wavy hair, wearing earrings and smiling." Several template variations are generated for each image in order to increase linguistic diversity. The captions are tokenised and converted into embeddings by a text encoder, producing a vector of fixed length that serves as the conditioning input for both the generator and the discriminator.

5.4 Conditional GAN

In a conditional GAN, the generator G maps a noise vector z and a text embedding c to an image, while the discriminator D evaluates an image x together with c. The training objective is the two-player minimax game expressed as follows:

```
min_G max_D V(D, G) = E_x [ log D(x | c) ] + E_z [ log (1 - D(G(z | c))) ]
```

To encourage agreement with the text, the discriminator is also presented with real images paired with mismatched captions, and it learns to classify such pairs as false. This matching-aware strategy was introduced by Reed et al. [3].

5.5 Training Procedure

Training alternates between the two networks, as shown in Figure 5.2. For every mini-batch, the generator produces fake images from noise and captions. The discriminator is then updated using real and fake images. Subsequently, the generator is updated so that the discriminator scores its images as real. Loss curves and sample images are monitored regularly, and hyper-parameters are adjusted if the training becomes unstable.

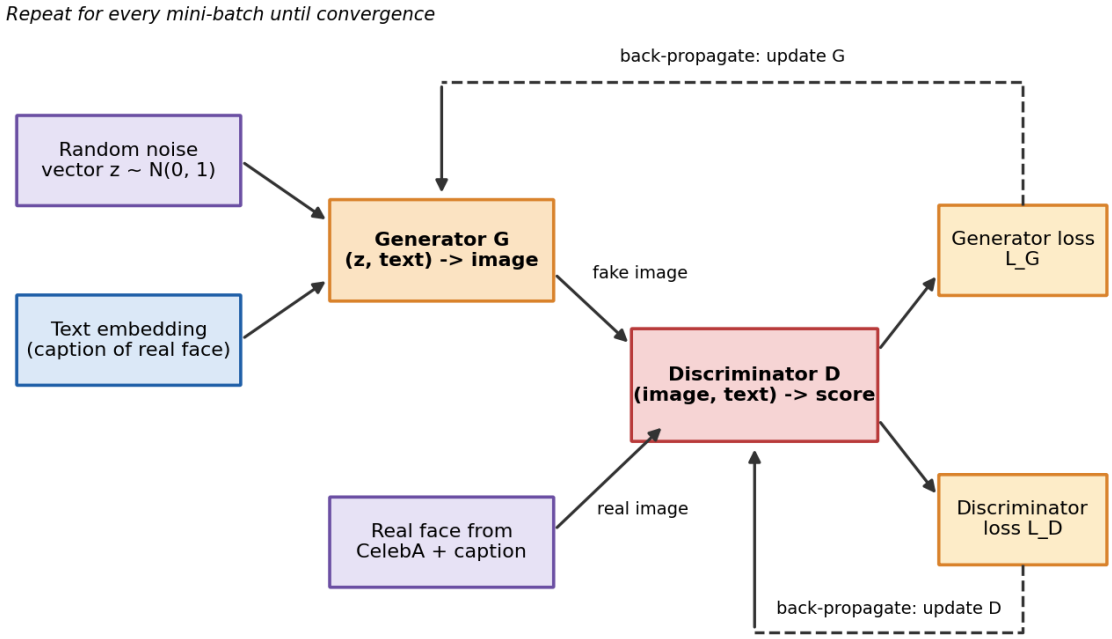


Figure 5.2: Adversarial training loop of the conditional GAN

5.6 Inference Procedure

During inference, only the generator is employed. The process depicted in Figure 5.3 begins when the user enters a description. An empty or invalid input produces an error message. Otherwise, the text is encoded, a random noise vector is sampled, the generator

produces the image, and the image is post-processed, displayed, and stored in the history. Sampling a new noise vector for the same text yields different faces that all satisfy the description.

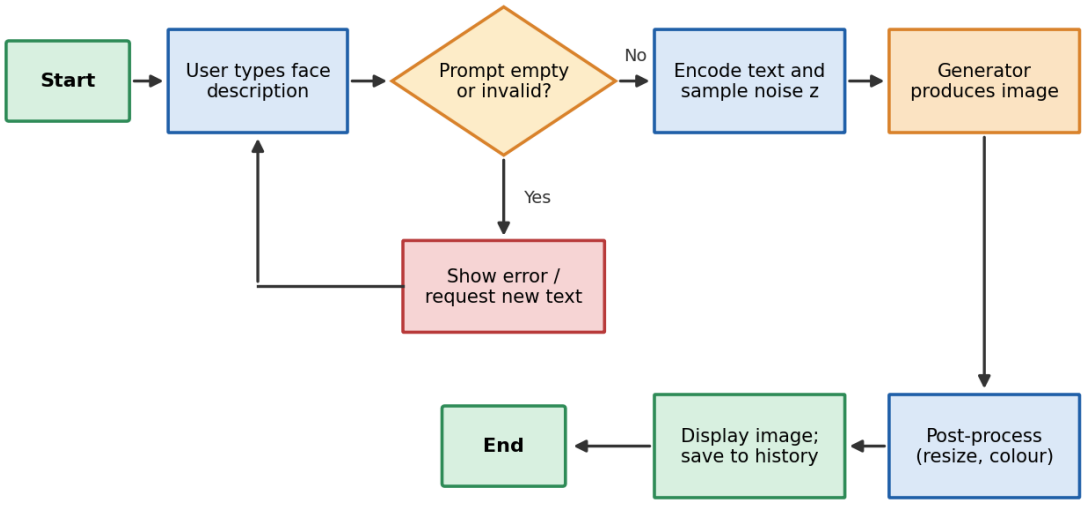


Figure 5.3: Flowchart of the inference (face generation) process

5.7 Evaluation Strategy

The generated images are evaluated with respect to two aspects: realism and semantic consistency. Realism is measured quantitatively using the Fréchet Inception Distance (FID) [30] and the Inception Score [31]. Lower FID values and higher Inception Scores indicate better quality. Text-image consistency is measured by the similarity between the prompt and the image, computed by a CLIP model [32]. This is supplemented by a small user study in which participants rate how well the face matches the description and how realistic it appears. A qualitative inspection of sample outputs completes the evaluation.

5.8 Deployment Plan

Deployment is conducted in phases. In the first phase, the trained model is wrapped in a Flask application that runs locally. In the second phase, the interface is refined with history, download, and print functions. In the third phase, the service can be containerised for use on a shared GPU server. Responsible-use notices are included throughout.

Chapter 6

Implementation

6.1 Introduction

This chapter describes how the design and the methodology were translated into a working prototype. It covers the development environment, dataset preparation, model training, the web application, and the generation back end.

6.2 Development Environment

Python is the principal development language because of its extensive ecosystem for machine learning. TensorFlow and PyTorch are used for deep-learning models, OpenCV for image processing, Hugging Face Transformers and Diffusers for text encoders and pre-trained pipelines, and Flask for the web server. The interface is written in HTML, CSS, and JavaScript. Development was carried out in Visual Studio Code within a Python virtual environment. Figure 6.1 shows the project in the editor, including the application file *app.py*, the dataset file, and the HTML page defined in the code.

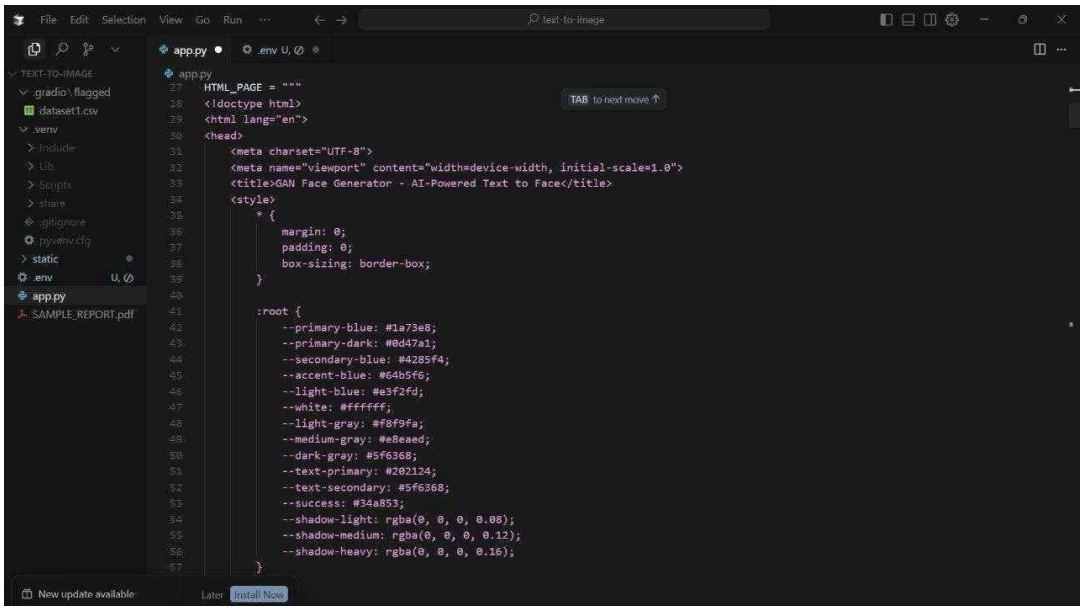


Figure 6.1: Project structure and application code in Visual Studio Code

The source code is organised into three logical components. The application module, contained in *app.py*, defines the Flask routes, loads the generation pipeline, and renders the HTML page. The static directory holds the stylesheet and the client-side scripts that control the prompt form and the result card. The dataset file records the attribute annotations used in the preparation of captions. Environment-specific settings, including the access token, are supplied through a separate environment file and are kept out of the source code.

6.3 Dataset Preparation

Dataset preparation consists of loading the CelebA images and the attribute file, cropping and resizing the faces to a fixed resolution, normalising the pixel values to [-1, 1], and generating the captions from the attributes. Random horizontal flipping is applied as a form of augmentation. The data is shuffled and supplied in mini-batches through a data loader.

6.4 Model Training

The generator and the discriminator are implemented as convolutional networks, as described in Section 4.4. Both networks are trained with the Adam optimiser and the adversarial loss defined in Chapter 5, with the discriminator and the generator updated in alternation. Checkpoints are saved at regular intervals, and sample grids of generated faces for fixed test prompts are written to disk so that the progress of training can be inspected visually.

6.5 Web Application

The web application is a Flask server. The main page, entitled *GAN Face Generator*, contains a text box labelled "Describe the face you want to generate", a Generate Face button, a result card, and a Generation History panel on the right (Figure 6.2). The result card provides download, save-to-history, and print actions. When the user submits a prompt, the browser transmits it to the server, which runs the generator and returns the image for display.

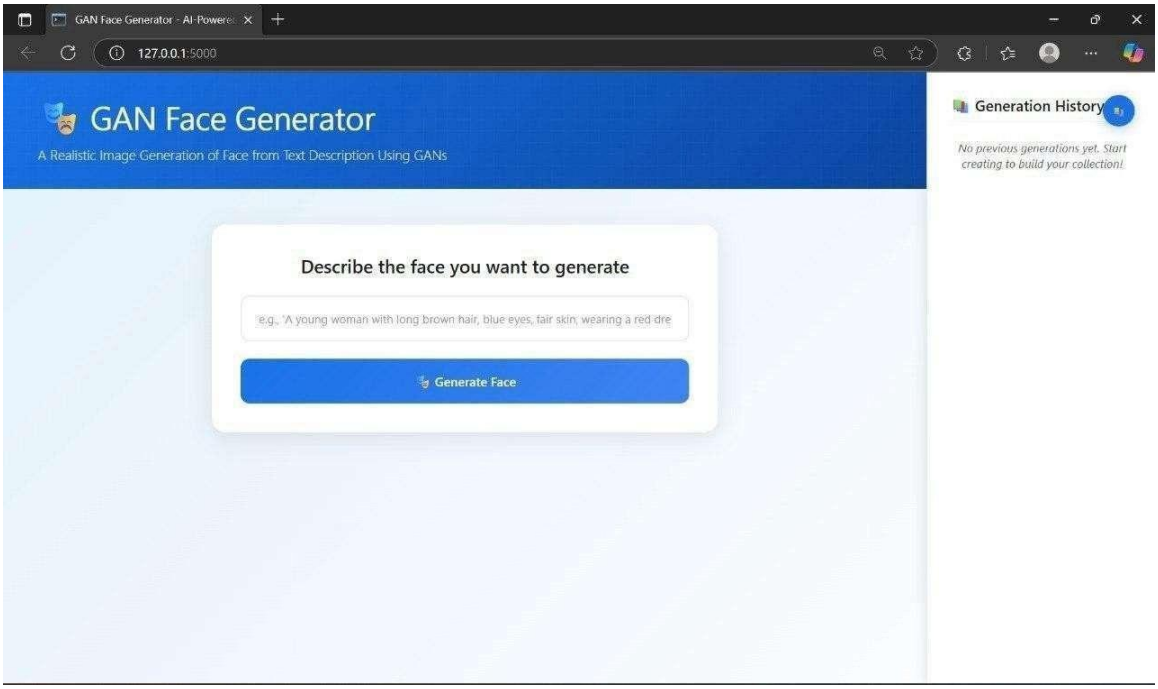


Figure 6.2: Home page of the GAN Face Generator web application

6.6 Generation Back End

At start-up, the server selects a CUDA-enabled GPU if one is available and otherwise falls back to the CPU. It loads the model pipeline once and then serves requests. In the current prototype, the code loads a pre-trained Stable Diffusion v1.5 pipeline through the Hugging Face Diffusers library [21], and it reads the access token from an environment variable rather than from the source code. The terminal output in Figure 6.3 shows that the model loaded successfully and that the Flask server started on the local address 127.0.0.1:5000. The conditional GAN described in Chapters 4 and 5 defines the architecture studied in this project. The prototype pipeline provides the face-generation engine behind the same interface.

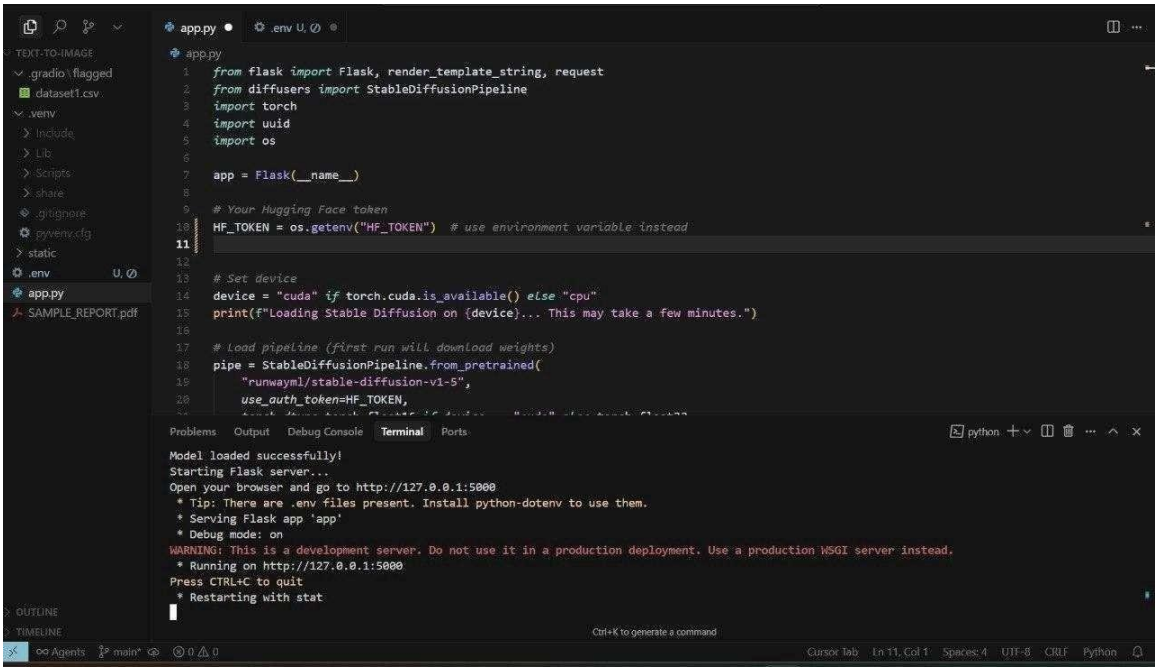


Figure 6.3: Model loading and Flask server start-up in the terminal

6.7 Sample Output

Figure 6.4 shows the interface after a prompt has been entered and a face has been generated. The generated image is displayed in the result card, with the download, save, and print buttons located beneath it.

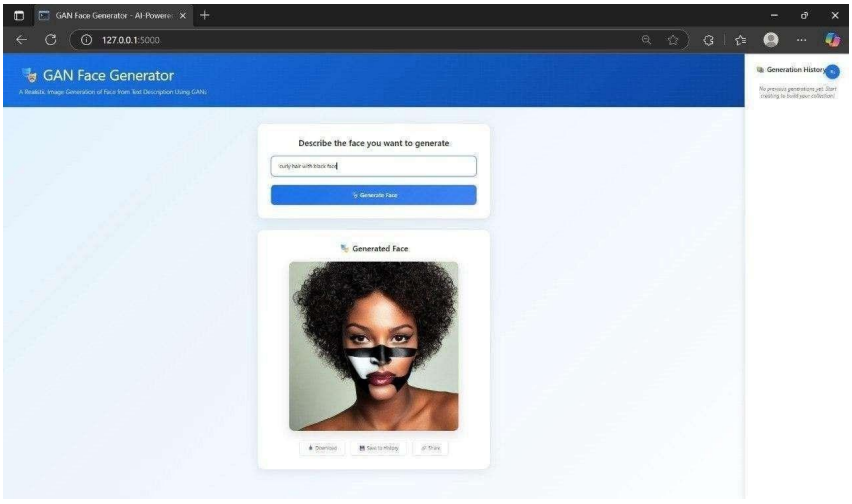


Figure 6.4: Generated face displayed in the web application

6.8 Implementation Challenges

Several challenges were encountered during implementation. Model loading requires considerable time on the first start. This is addressed by loading the model once at start-up and displaying a progress message in the terminal. Execution on the CPU is substantially slower than on a GPU, so the device is selected automatically. Access credentials must not be hard-coded, and they are therefore read from environment variables. Finally, generated faces may contain artefacts in fine details, such as the eye region. This limitation is addressed in the evaluation and future-work sections.

6.9 Summary

The implementation delivers a working web application in which a user enters a description and receives a generated face. The modular structure keeps the interface, the text handling, and the generation engine separate. Consequently, the engine can be replaced by the trained conditional GAN, or by an improved model, without altering the interface.

Chapter 7

Testing

7.1 Introduction

This chapter describes the testing approach adopted for the system. It covers the test strategy, the functional test cases, the quantitative and qualitative evaluation of the generated images, the observations made from the prototype, and the limitations of the study.

7.2 Test Strategy

Testing is organised into three levels. Unit testing examines individual components, such as text pre-processing and image post-processing. Integration testing verifies that the interface, the text encoder, and the generator operate together correctly. System testing examines the complete flow, from the entry of a description to the display of the generated face. The tests are repeated on both GPU and CPU configurations.

7.3 Functional Test Cases

Table 7.1: Functional test cases

ID	Module	Test case	Expected result	Status
TC-01	Start-up	Run the application and load the model	The model loads and the server starts on 127.0.0.1:5000	Pass
TC-02	Interface	Open the home page in a browser	The title, prompt box, button, and history panel are displayed	Pass
TC-03	Text input	Enter a face description	The text is accepted in the prompt box	Pass
TC-04	Generation	Click Generate Face	A face image is displayed in the result card	Pass

Each test case listed in Table 7.1 was verified from the execution of the application shown in Chapter 6, where the model loaded, the interface opened, a prompt was accepted, and a

face was generated.

7.4 Image Quality Evaluation

The quality of the generated faces will be measured using the metrics listed in Table 7.2. Quantitative scores will be computed on a fixed set of test prompts, using the same number of generated images for every model under comparison, so that the results are directly comparable.

Table 7.2: Evaluation metrics

Metric	What it measures	Preferred value
FID [30]	Distance between the feature statistics of real and generated faces	Lower
Inception Score [31]	Clarity and diversity of the generated images	Higher
CLIP similarity [32]	Agreement between the prompt and the generated face	Higher
User rating	Perceived realism and correspondence with the description (1 to 5 scale)	Higher

7.5 Observations from the Prototype

In the sample run illustrated in Figure 6.4, the generated portrait is sharp, and the overall hair texture, skin tone, and lighting appear natural. A visible artefact is present across the eye region, however, which indicates that fine facial details remain a weakness of the approach. This observation corresponds with the literature, which notes that even advanced models occasionally fail to capture fine-grained facial details [20], [21]. The results also depend on the wording of the prompt. Descriptions that specify concrete attributes, such as hair colour, age, and accessories, yield more reliable results than vague descriptions.

7.6 Limitations of the Study

- The test set of prompts is small, and the findings therefore cannot yet be generalised.
- The quantitative metrics (FID, Inception Score, and CLIP similarity) and the user study have yet to be completed on a larger scale.
- Rare attributes, such as scars and tattoos, are not covered.

- The training data is not demographically balanced, which may affect quality across different groups.
- The output quality and the speed of generation depend on the available GPU.

7.7 Summary

The functional tests verified the principal flow from the entry of a prompt to the display of a generated face. A structured plan for quantitative and user-based evaluation has been defined. The observations made from the prototype demonstrate both the strengths of the approach, namely realistic texture and lighting, and its principal weakness, namely fine facial detail.

Chapter 8
Conclusion

8.1 Conclusion

This project studied and implemented a system that generates realistic human face images from natural-language descriptions. Beginning with a survey of 20 significant works, the report identified a gap between text-conditioned models, which exhibited limited realism, and face-specific models, which offered high realism but no text control. The proposed design combines a text encoder with a conditional GAN, trained on image-caption pairs derived from CelebA, and it exposes the result through an easy-to-use web interface.

The prototype demonstrates the complete flow from a typed description to a displayed face, together with generation history and export options. Four functional test cases, covering start-up, the interface, text input, and image generation, were verified from the execution of the application. The sample output exhibited sharp texture, a natural skin tone, and good lighting. At the same time, the sample revealed a visible artefact in the eye region, which confirms the observation in the literature that fine facial detail is the most difficult aspect of face synthesis. The observations also indicated that prompts naming concrete attributes, such as hair colour, age, and accessories, produce more reliable results than vague prompts.

It is important to state the present status of the work with precision. The architecture, data flow, training methodology, and evaluation plan of the conditional GAN have been fully designed and documented. The working prototype, however, currently employs a pre-trained Stable Diffusion v1.5 pipeline as its generation engine behind the same web interface. Because the interface, the text handling, and the generation engine are separate modules, the engine can be replaced by the trained conditional GAN without redesigning the application. Training the GAN on CelebA captions, and computing the quantitative metrics therefore constitute the principal steps that remain.

The evaluation plan, which is based on FID, the Inception Score, CLIP similarity, and user ratings, provides a structured means of measuring realism and semantic accuracy. The work also highlights practical issues, including the stability of GAN training, dataset bias, and the ethical necessity of labelling synthetic faces.

8.2 Achievement of Objectives

Table 8.1 relates the outcomes of the project to the objectives set out in Chapter 1.

Table 8.1: Project objectives and their status

Objective	Outcome	Status
Study and analyse existing GAN-based text-to-image methods	A survey of 20 works covering GANs, multi-stage and attention-based GANs, face GANs, and diffusion models, with a comparison table and the identified research gap	Completed
Design and implement a GAN framework for face generation from text	The conditional GAN architecture, data flow, UML diagrams, and training methodology have been designed. The prototype currently operates on a pre-trained Stable Diffusion pipeline.	Designed; GAN training pending
Evaluate the generated images with respect to realism and semantic accuracy	The evaluation plan (FID, Inception Score, CLIP similarity, and user rating) has been defined. Sample output has been inspected qualitatively.	Partly completed
Provide an interactive interface for end users	A Flask web application comprising a prompt box, a result card, history, and download and print actions	Completed

8.3 Key Contributions

1. A survey and comparison of 20 text-to-image and face-generation works, with the research gap identified.
2. A design for text-conditioned face generation using a conditional GAN, presented through architecture, data-flow, and UML diagrams.
3. A methodology covering dataset preparation, caption generation, adversarial training, inference, and evaluation.
4. An interactive web application, the GAN Face Generator, with prompt input, result display, history, and download and print options.
5. A test plan comprising functional test cases and quality metrics, together with a discussion of ethical considerations.

8.4 Limitations Observed

The principal limitations of the present work are summarised below in order that the results may be interpreted in the appropriate context.

- The trained conditional GAN is not yet part of the prototype. The current engine is a pre-trained diffusion pipeline.
- The functional test suite covers the principal flow from prompt to image, and the prompt set is small.
- The FID, the Inception Score, the CLIP similarity, and the user study have not yet been computed.
- Fine facial details, such as the eye region, may contain artefacts.
- CelebA is not demographically balanced, and rare attributes, such as scars and tattoos, are not covered.

8.5 Ethical and Social Perspective

Realistic face generation carries a responsibility to prevent misuse. The system generates only synthetic faces, accepts no photographs of real persons, and retains no personal data beyond the local history. When employed for forensic purposes, a generated face constitutes an investigative aid and must never be regarded as proof of identity. Balanced training data, results reported across attribute groups, and visible or invisible "AI-generated" labels are necessary before wider deployment. These measures ensure that the technology remains fair and transparent for all of its users.

8.6 Future Enhancements

- Train the complete conditional GAN on a larger and more balanced face dataset, and report the FID and CLIP scores.
- Extend the functional test suite and complete the user study.
- Support rare attributes, such as scars, tattoos, and facial hair styles.
- Adopt a style-based generator to achieve higher resolution and finer control.
- Permit the interactive editing of a generated face through modifications to the text.
- Introduce watermarking and responsible-use safeguards.

8.7 Final Remarks

The project demonstrates that describing a face in words and receiving a realistic image is achievable with current deep-learning methods, and that such a capability can serve forensic, entertainment, and research purposes. The literature survey, the design, the working prototype, and the test and evaluation plan together form a sound foundation for continued work. With further training data, evaluation, and safeguards, the system can develop into a reliable tool for text-driven face synthesis.

Chapter 9

Future Work

This chapter describes the principal directions for future work. Each direction responds to a limitation identified in Chapters 7 and 8, and the roadmap is summarised in Table 9.1.

9.1 Stronger Text Understanding

The text encoder is the component that most directly determines whether a generated face corresponds to the description. Future versions will employ larger pre-trained language encoders so that long, detailed, or loosely worded descriptions are interpreted correctly. The present prototype is sensitive to the phrasing of the prompt, and this sensitivity can be reduced by training on a broader range of paraphrases for each attribute.

Handling negation represents an important next step. A description such as "no beard" presently risks producing a face with a beard, because the presence of the word may be interpreted as the presence of the attribute. Relations between attributes, such as "a man with a beard wearing glasses" or "a woman with long hair tied back", require the model to associate each attribute with the correct region of the face. Evaluation of these cases will require a dedicated test set of negated and relational prompts.

9.2 Rare and Fine-Grained Attributes

The current scope covers basic features only. Extending the dataset and the captions to include scars, tattoos, freckles, birthmarks, and detailed facial-hair styles would render the system considerably more useful for forensic sketching. Such attributes are seldom annotated in CelebA, and their inclusion will therefore require either additional labelled data or a semi-automatic annotation process verified by human reviewers.

9.3 Higher Resolution and Realism

Style-based generators, such as StyleGAN2 [15], together with progressive training [13], can raise the output resolution to 1024 × 1024 and reduce artefacts of the kind observed around the eyes in the sample output. Combining the strengths of GANs and diffusion

models is another promising direction. A hybrid architecture may use a GAN for rapid generation of structure and a diffusion stage for refinement of texture and fine detail.

9.4 Interactive Editing and Refinement

Users should be able to refine a generated face progressively. For example, a user may change the hair colour or add glasses through a follow-up sentence, while the identity of the face remains stable. Latent-space editing along attribute directions provides a practical mechanism for this purpose. Each attribute direction is obtained from the latent space, and the generated face is moved along that direction by an amount controlled by the user.

9.5 Fairness, Bias, and Responsible Use

Because CelebA is not demographically balanced, the quality of the generated faces may vary across age, gender, and skin-tone groups. Future work will train on more balanced data, apply stratified sampling, and report results separately for each attribute group. Such reporting is essential in order to identify the groups for which the system performs poorly.

Watermarks or visible labels should be applied to every generated image, so that synthetic faces cannot be presented as photographs of real persons. Clear usage guidelines are required for forensic applications, and these guidelines should state explicitly that a generated face is not evidence of identity.

9.6 Multilingual Prompts

Many users describe faces in their own language. Supporting Indian languages, including Hindi, Telugu, and Gujarati, through multilingual text encoders would make the tool considerably more accessible. This extension requires parallel caption data in each language and an evaluation procedure that verifies the correspondence between a description and the resulting face in every supported language.

9.7 Performance and Deployment

Model compression, mixed-precision inference, and containerised deployment on a shared GPU server would reduce the time required for generation and permit a larger number of users to use the system simultaneously. Caching frequently requested prompts and their embeddings would also reduce the computational cost of repeated requests.

9.8 Validation with Domain Experts

For forensic use, the generated faces should be evaluated together with trained sketch artists and investigators. Such an evaluation will measure how effectively the outputs support genuine identification tasks, as opposed to how realistic they appear. A structured study involving domain experts would provide evidence that is not available from automated metrics alone.

9.9 Beyond Static Images

Longer-term directions include multi-view generation, three-dimensional face models, and text-driven animated avatars.

9.10 Summary of Future Directions

Table 9.1: Planned future work and priorities

Area	Planned action	Priority
Text understanding	Adopt larger language encoders; add negation and relation test sets	High
Rare attributes	Extend captions and data for scars, tattoos, and facial hair	Medium
Resolution and realism	Evaluate style-based and hybrid architectures with FID and CLIP scores	High
Interactive editing	Implement attribute-direction editing with identity-preservation measures	Medium
Fairness and responsible use	Balance data, report results by group, and add watermarks	High
Multilingual prompts	Add Hindi, Telugu, and Gujarati captions and verify correspondence	Low
Performance and deployment	Apply compression, containerisation, and latency monitoring	Medium
Expert validation	Conduct forensic evaluation with sketch artists and investigators	High
Beyond static images	Explore multi-view generation, three-dimensional models, and animation	Low

References

[1] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, "Generative adversarial nets," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2014. [Online]. Available: https://arxiv.org/abs/1406.2661

[2] A. Radford, L. Metz, and S. Chintala, "Unsupervised representation learning with deep convolutional generative adversarial networks," in *Proc. ICLR*, 2016. [Online]. Available: https://arxiv.org/abs/1511.06434

[3] S. Reed, Z. Akata, X. Yan, L. Logeswaran, B. Schiele, and H. Lee, "Generative adversarial text to image synthesis," in *Proc. ICML*, 2016. [Online]. Available: https://arxiv.org/abs/1605.05396

[4] S. Reed, Z. Akata, B. Schiele, and H. Lee, "Learning deep representations of fine-grained visual descriptions," in *Proc. IEEE CVPR*, 2016. [Online]. Available: https://arxiv.org/abs/1605.05395

[5] H. Zhang, T. Xu, H. Li, S. Zhang, X. Wang, X. Huang, and D. Metaxas, "StackGAN: Text to photo-realistic image synthesis with stacked generative adversarial networks," in *Proc. IEEE ICCV*, 2017. doi: 10.1109/ICCV.2017.629

[6] H. Zhang, T. Xu, H. Li, S. Zhang, X. Wang, X. Huang, and D. Metaxas, "StackGAN++: Realistic image synthesis with stacked generative adversarial networks," *IEEE Trans. Pattern Anal. Mach. Intell.*, 2018. doi: 10.1109/TPAMI.2018.2856256

[7] T. Xu, P. Zhang, Q. Huang, H. Zhang, Z. Gan, X. Huang, and X. He, "AttnGAN: Fine-grained text to image generation with attentional generative adversarial networks," in *Proc. IEEE/CVF CVPR*, 2018.

[8] M. Zhu, P. Pan, W. Chen, and Y. Yang, "DM-GAN: Dynamic memory generative adversarial networks for text-to-image synthesis," in *Proc. IEEE/CVF CVPR*, 2019.

[9] T. Qiao, J. Zhang, D. Xu, and D. Tao, "MirrorGAN: Learning text-to-image generation by redescription," in *Proc. IEEE/CVF CVPR*, 2019. [Online]. Available: https://arxiv.org/abs/1903.05854

[10] Y. Li, Z. Gan, Y. Shen, J. Liu, Y. Cheng, Y. Wu, L. Carin, D. Carlson, and J. Gao, "StoryGAN: A sequential conditional GAN for story visualization," in *Proc. IEEE/CVF CVPR*, 2019. [Online]. Available: https://arxiv.org/abs/1812.02784

[11] S. Nam, Y. Kim, and S. J. Kim, "Text-adaptive generative adversarial networks: Manipulating images with natural language," in *Advances in Neural Information Processing Systems*, 2018. [Online]. Available: https://arxiv.org/abs/1810.11919

[12] J.-Y. Zhu, P. Krähenbühl, E. Shechtman, and A. A. Efros, "Generative visual manipulation on the natural image manifold," in *Proc. ECCV*, 2016. [Online]. Available: https://arxiv.org/abs/1609.03552

[13] T. Karras, T. Aila, S. Laine, and J. Lehtinen, "Progressive growing of GANs for improved quality, stability, and variation," in *Proc. ICLR*, 2018. [Online]. Available: https://arxiv.org/abs/1710.10196

[14] T. Karras, S. Laine, and T. Aila, "A style-based generator architecture for generative adversarial networks," in *Proc. IEEE/CVF CVPR*, 2019.

[15] T. Karras, S. Laine, M. Aittala, J. Hellsten, J. Lehtinen, and T. Aila, "Analyzing and improving the image quality of StyleGAN," in *Proc. IEEE/CVF CVPR*, 2020.

[16] A. Brock, J. Donahue, and K. Simonyan, "Large scale GAN training for high fidelity natural image synthesis," in *Proc. ICLR*, 2019. [Online]. Available: https://arxiv.org/abs/1809.11096

[17] J.-Y. Zhu, T. Park, P. Isola, and A. A. Efros, "Unpaired image-to-image translation using cycle-consistent adversarial networks," in *Proc. IEEE ICCV*, 2017. [Online]. Available: https://arxiv.org/abs/1703.10593

[18] A. Ramesh, M. Pavlov, G. Goh, S. Gray, C. Voss, A. Radford, M. Chen, and I. Sutskever, "Zero-shot text-to-image generation," in *Proc. ICML*, 2021. [Online]. Available: https://arxiv.org/abs/2102.12092

[19] A. Nichol, P. Dhariwal, A. Ramesh, P. Shyam, P. Mishkin, B. McGrew, I. Sutskever, and M. Chen, "GLIDE: Towards photorealistic image generation and editing with text-guided diffusion models," in *Proc. ICML*, 2022. [Online]. Available: https://arxiv.org/abs/2112.10741

[20] C. Saharia et al., "Photorealistic text-to-image diffusion models with deep language understanding," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. [Online]. Available: https://arxiv.org/abs/2205.11487

[21] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-resolution image synthesis with latent diffusion models," in *Proc. IEEE/CVF CVPR*, 2022. [Online]. Available: https://arxiv.org/abs/2112.10752

[22] O. R. Nasir, S. K. Jha, M. S. Grover, Y. Yu, A. Kumar, and R. R. Shah, "Text2FaceGAN: Face generation from fine grained textual descriptions," in *Proc. IEEE BigMM*, 2019. doi: 10.1109/BigMM.2019.00-42

[23] M. Oza, S. Chanda, and D. Doermann, "Semantic text-to-face GAN: ST2FG," *arXiv preprint*, 2021. [Online]. Available: https://arxiv.org/abs/2107.10756

[24] P. Yadav, A. Singh, and S. Sharma, "Text-to-face generation using conditional GANs," in *Proc. Int. Conf. Intelligent Systems and Data Science (ICISDS)*, India, 2022.

[25] R. Patel, A. Kumbhare, and S. Bhope, "Facial image synthesis from natural language descriptions using deep convolutional GANs," *Int. J. Eng. Res. Technol. (IJERT)*, vol. 10, no. 6, 2021.

[26] Y. Cao, J. Qin, Q. Ma, H. Sun, K. Yan, L. Wang, and J. Ren, "Survey of text-to-image synthesis," *J. Zhejiang Univ.*, 2024.

[27] S. S. Sawant, S. Kaliraj, S. Raghavendra, and R. Shirwaikar, "Exploring the evolution of generative adversarial network architectures for text to image synthesis: A comprehensive review," *Discover Artificial Intelligence*, vol. 6, p. 1027, 2026.

[28] A. H. Bermano et al., "State-of-the-art in the architecture, methods and applications of StyleGAN," *Computer Graphics Forum*, vol. 41, no. 2, pp. 591–611, 2022.

[29] Z. Liu, P. Luo, X. Wang, and X. Tang, "Deep learning face attributes in the wild," in *Proc. IEEE ICCV*, 2015. [Online]. Available: https://arxiv.org/abs/1411.7766

[30] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, and S. Hochreiter, "GANs trained by a two time-scale update rule converge to a local Nash equilibrium," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. [Online]. Available: https://arxiv.org/abs/1706.08500

[31] T. Salimans, I. Goodfellow, W. Zaremba, V. Cheung, A. Radford, and X. Chen, "Improved techniques for training GANs," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2016. [Online]. Available: https://arxiv.org/abs/1606.03498

[32] A. Radford et al., "Learning transferable visual models from natural language supervision," in *Proc. ICML*, 2021. [Online]. Available: https://arxiv.org/abs/2103.00020

[33] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, 2019. [Online]. Available: https://arxiv.org/abs/1810.04805

Appendix A: Installation and Deployment Guide

This appendix summarises the procedure for installing and running the GAN Face Generator web application on a development machine. Exact package versions and file names may differ slightly in the project repository.

A.1 Prerequisites

- Python 3.10 or later, together with pip and the venv module.
- A Hugging Face account and an access token, which are required to download the pre-trained model weights.
- Optionally, but recommended: an NVIDIA GPU with CUDA support and 6 GB or more of VRAM. Without a GPU, the application runs on the CPU and is considerably slower.
- A modern web browser and approximately 20 GB of free disk space for the model files.

A.2 Python Packages and Environment Variable

Table A.1: Required packages and configuration value

Item	Purpose
flask	Web server and page rendering for the user interface
torch	Deep-learning framework used to run the model on a GPU or the CPU
diffusers, transformers	Hugging Face libraries used to load and run the generation pipeline
HF_TOKEN (environment variable)	Hugging Face access token read by the application at start-up; it is not stored in the source code

A.3 Start-up Steps

1. Copy the project folder to the target machine and open it in a terminal or in Visual Studio Code.
2. Create and activate a virtual environment.
3. Install the required packages using pip.
4. Set the HF_TOKEN environment variable to the Hugging Face access token.
5. Run the application with the command *python app.py*.
6. Wait for the message "Model loaded successfully". The first start may require several minutes because the model files must be downloaded.
7. Open http://127.0.0.1:5000 in a browser, enter a face description, and select Generate Face.

Listing A.1: Typical installation and start-up commands

```
python -m venv venv
venv\Scripts\activate
pip install flask torch diffusers transformers
set HF_TOKEN=your_token_here
python app.py
```

A.4 Troubleshooting

Table A.2: Common problems and remedies

Problem	Probable cause and remedy
Very slow first start	The model files are being downloaded. Allow the download to complete; subsequent starts are faster.
Generation takes a long time	The application is running on the CPU. Install a CUDA-enabled version of PyTorch and verify that the GPU is detected.
Authentication or download error	The HF_TOKEN variable is missing or invalid. Set it again and restart the application.
Page does not open	The server is not running, or port 5000 is in use. Check the terminal output and restart the server.

Appendix B: Sample Prompt Templates

The following templates and examples illustrate how face descriptions may be written for the system. Placeholders enclosed in square brackets are replaced by the user with concrete attribute values. Descriptions that name specific attributes produce more reliable results than vague descriptions.

Listing B.1: Face description template

```
A [age group] [gender] with [hair colour], [hair style] hair,
[eye colour] eyes and [skin tone] skin. The person has
[facial hair] and is wearing [accessories]. The expression is
[expression]. Photorealistic portrait, front view, soft lighting.
```

B.1 Example Prompts

Table B.1: Example face descriptions

No.	Example description
1	A young woman with long brown hair, blue eyes, and fair skin, wearing a red dress.
2	An old man with short grey hair, a white beard, and round glasses, smiling.
3	A middle-aged man with black curly hair, brown eyes, and a trimmed moustache.
4	A young woman with blond wavy hair, wearing earrings and light makeup.
5	A man with a bald head, dark skin, and a serious expression.

B.2 Attribute-to-Caption Mapping for Training

During training, captions are generated from the CelebA attribute labels. Table B.2 shows how typical attributes are converted into phrases, which are then combined into a complete sentence.

Table B.2: CelebA attributes and caption phrases

CelebA attribute	Phrase used in caption
Young	young (otherwise: older)
Male	man (otherwise: woman)
Blond_Hair, Brown_Hair, Black_Hair, Gray_Hair	blond, brown, black, or grey hair
Wavy_Hair, Straight_Hair, Bangs	wavy hair, straight hair, or bangs
Eyeglasses	wearing glasses
Smiling	smiling
Wearing_Hat, Wearing_Earrings	wearing a hat, or wearing earrings
Heavy_Makeup, Wearing_Lipstick	wearing makeup, or wearing lipstick

Appendix C: Glossary

Table C.1 defines the principal technical terms employed in this report.

Table C.1: Glossary of key terms

Term	Meaning
GAN	Generative Adversarial Network: a generator and a discriminator trained in opposition to each other
Generator	A network that creates images from a noise vector and, in this project, a text embedding
Discriminator	A network that determines whether an image is real or generated
Conditional GAN	A GAN that receives additional information, such as text, to control its output
Latent vector	A vector of numbers from which the generator creates an image
Text embedding	A numerical vector representing the meaning of a sentence
Prompt	The text description entered by the user
CelebA	A large dataset of celebrity face images with 40 attribute labels
Mode collapse	A GAN failure in which the generator produces only a small number of similar outputs
FID	Fréchet Inception Distance: the distance between the statistics of real and generated images; lower is better
Inception Score	A measure of the clarity and diversity of generated images; higher is better
CLIP	A model that scores how well an image corresponds to a text description
Diffusion model	A model that generates images by progressively removing noise
Batch normalisation	A technique that stabilises and accelerates the training of neural networks
Flask	A lightweight Python framework used for the web interface
Artefact	An unwanted visual error in a generated image